

CSS PART 2

Some of the common CSS properties include the following:

- color: the color of text
- text-align: where elements are placed on the page
- background-color: can be set to any color
- width: in pixels or percent of a page
- height: in pixels or percent of a page
- padding: how much space should be left inside an element
- margin: how much space should be left outside an element
- font-family: type of font for text on page
- font-size: in pixels
- border: size type (solid, dashed, etc) color

CSS SELECTORS

There's a good introduction into CSS selectors. Please read more here

(https://www.w3schools.com/cssref/css_selectors.asp)

We are able to determine which HTML element we are styling by using the

- **element type:** this includes what we have done so far: styling all elements of the same type
- **id:** this involves giving our HTML elements an id like
`<h1 id="firstHeader">Hello!</h1>`

and then apply the styling using `#firstHeader {....}`. The hashtag indicates that we are searching by id. No two elements can have the same id, and no element can have more than one id.

- **class:** This is similar to id, but a class can be shared by more than one element, and a single element can have more than one class. We add classes to an HTML element like this: `<h1 class="page-text muted">Hello!</h1>` (note that we just added two classes to the element: page-text and muted). We then style based on class using a period instead of a hashtag: `.muted {...}`

Now, we also have to deal with the problem of potentially conflicting CSS. What happens when a header should be red based on its class but blue based on its id? CSS has a specificity order that goes:

1. In-line styling
2. id
3. class
4. element type

In addition to the comma for multiple selectors, several other ways exist to specify which elements you want to style. This table provides a few, and we'll go through a few examples below:

a, b	Multiple Element Selector
a b	Descendant Selector
a > b	Child Selector
a + b	Adjacent Sibling Selector
[a=b]	Attribute Selector
a:b	Pseudoclass Selector
a::b	Pseudoelement Selector

Descendant Selector: Here, we use the descendant selector only to apply styling to list items found within an unordered list:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Using Selectors</title>
    <style>
      ul li {
        color: blue;
      }
    </style>
  </head>
  <body>
    <ol>
      <li>foo</li>
      <li>bar
        <ul>
          <li>hello</li>
          <li>goodbye</li>
          <li>hello</li>
        </ul>
      </li>
      <li>baz</li>
    </ol>
  </body>
</html>
```

Attributes as Selectors: We can also narrow our selection based on the attributes we assign to HTML elements using brackets. For example, in the following list of links, we choose to only make the link to Amazon red:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Using Selectors</title>
    <style>
      a[href="https://www.amazon.com/"] {
        color: red;
      }
    </style>
  </head>
  <body>
    <a href="https://www.amazon.com/">Amazon</a>
  </body>
</html>
```

```

    }
  </style>
</head>
<body>
  <ol>
    <li><a href="https://www.google.com/">Google</a></li>
    <li><a href="https://www.amazon.com/">Amazon</a> </li>
    <li><a href="https://www.facebook.com/">Facebook</a></li>
  </ol>
</body>
</html>

```

Not only can we use CSS to change what an element looks like permanently, but also what it looks like under certain conditions. For example, what if we wanted a button to change colour when we hover over it? We can achieve this using a CSS pseudoclass, (https://www.w3schools.com/css/css_pseudo_classes.asp) which provides additional styling during special circumstances. We write this by adding a colon after our selector, and then adding the circumstance after that colon.

In the case of the button, we would add **:hover** to the button selector to specify the design only when hovering:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Pseudoclasses</title>
    <style>
      button {
        background-color: red;
        width: 200px;
        height: 50px;
        font-size: 24px;
      }

      button:hover {
        background-color: green;
      }
    </style>
  </head>
  <body>
    <button>Button 1</button>
    <button>Button 2</button>
    <button>Button 3</button>
  </body>
</html>

```

LISTS

Lists allow the grouping of items on a web page. The two ways of listing items are using an unordered list tag (which uses bullets by default) and the ordered list tag (which uses numbers by default).

Unordered list

An unordered list is created using the `<ul>` tag, and each list item starts with the `<li>` tag.

```
<ul>Vegetables I like
  <li>Carrots</li>
  <li>Cabbage</li>
  <li>Onions</li>
  <li>Lentils</li>
</ul>
```

The CSS **list-style-type** property is used to define the style of the list item marker. It can have values such as **disc**, **circle**, **square** or **none**.

```
<ul style="list-style-type: disc;">Vegetables I like
  <li>Carrots</li>
  <li>Cabbage</li>
  <li>Onions</li>
  <li>Lentils</li>
</ul>
```

Ordered List

An ordered list is created with the `<ol>` tag, and each list item starts with the `<li>` tag.

```
<ol>Vegetables I like
  <li>Carrots</li>
  <li>Cabbage</li>
  <li>Onions</li>
  <li>Lentils</li>
</ol>
```

The `<type>` attribute of the `<ol>` tag defines the type of the list item marker. The values that the **type** can take are:

type="1" – the list will be numbered with numbers (this is the default setting)

type="a" – the list will be numbered with lowercase letters

type="A" – the list will be numbered with uppercase letters

type="i" = the list will be numbered with lowercase roman numbers

type="I" – the list will be numbered with uppercase roman numbers

NOTE: Lists can be nested.

Example:

```

<ul>
  <li>Full Name</li>
  <li>First Name
    <ul>
      <li>Ishmael</li>
      <li>Gloria</li>
    </ul>
  </li>
  <li>Surname</li>
</ul>

```

!!!Students to read!

Horizontal list with CSS

DESCRIPTION LISTS

A description list allows each item in a list to have a description. The **<dl>** tag defines the description list, the **<dt>** tag defines the term (or name) and the **<dd>** tag describes each term.

```

<dl>
  <dt>Carrots</dt>
  <dd>- It a root vegetable.</dd>
  <dt>Cabbage</dt>
  <dd>- It is a leafy green plant</dd>
  <dt>Lentils</dt>
  <dd>- It is a legume with lens-shaped edible seeds</dd>
</dl>

```

BUTTONS

The **<button>** tag can be used to create buttons on the HTML page. Buttons can be styled by setting several different attributes. The Pseudo class **:Hover** can be used to change styles on hovering over the button.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Buttons</title>
  <style>
    button {
      width: 100px;
      height: 50px;
      background-color: black;
      color: yellow;
      padding: 10px;
      border-radius: 5px;
      font-size: 18;
    }

    button:hover {
      width: 105px;
      height: 70px;
      background-color: rgb(244, 3, 3);
      color: white;
      padding: 10px;
      border-radius: 10px;
      font-size: 24;
    }
  </style>
</head>
<body>
  <button>Click Me!</button>
</body>
</html>
```

FORMS

Forms allow a website to collect information from users. A form can contain several different types of input

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Forms</title>
</head>
<body>
  <form>
    <input type="text" placeholder="First Name" name="first">
    <input type="password" placeholder="Password" name="password">
    <br/>

    <!--Using radio buttons-->

    Favourite Colour:
    <input name="color" type="radio" value="blue">Blue
    <input name="color" type="radio" value="green">Green
    <input name="color" type="radio" value="yellow">Yellow
    <input name="color" type="radio" value="pink">Pink
    <br/>
    <!--Using checkboxes-->

    Fruits you likes (select all that apply) <br/>
    <input name="fruit" type="checkbox" value="Apple">Apple
    <input name="fruit" type="checkbox" value="Banana">Banana
    <input name="fruit" type="checkbox" value="Pear">Pear
    <input name="fruit" type="checkbox"
value="Strawberry">Strawberry
    <input name="fruit" type="checkbox" value="Pawpaw">Pawpaw
    <br/>
    <input type="Submit">
    <input type="Reset">

  </form>
</body>
</html>
```

THE <DIV> ELEMENT

The <div> element is a block element by default. It can contain other elements within it and can be styled to different outlooks.

```
<div>
  <h2>KNUST</h2>
  <p>The number one destination of choice.</p>
  <p>Cleanest and most serene environment.</p>
</div>
```

The <div> can be styled with different CSS properties.

```
<!DOCTYPE html>
<html>
<style>
  div.mydiv {
    width:100%;
    overflow:auto;
  }
  div.mydiv div {
    width:33%;
    float:left;
  }
</style>
<body>

<div class="mydiv">

  <div style="background-color:hsl(299, 92%, 50%);">
    <h2>KNUST</h2>
    <p>The number one destination of choice.</p>
    <p>Cleanest and most serene environment.</p>
  </div>

  <div style="background-color:#670d18; color:white">
    <h2>KNUST</h2>
    <p>The number one destination of choice.</p>
    <p>Cleanest and most serene environment.</p>
  </div>

  <div style="background-color:hsl(103, 38%, 89%);">
    <h2>KNUST</h2>
```



```
<p>The number one destination of choice.</p>
<p>Cleanest and most serene environment.</p>
</div>

</div>
```

RESPONSIVE DESIGN

It is important to make sure your website is readable to people on all types of devices. The **Viewport**, which is the part of the screen that is actually visible to the user at any given time, can be adapted to respond to varied device sizes. One way is to add this line to the head section of your page:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Another way is to use media queries which allow styles of a page to be changed based on the size of the device.

Example:

Let us change the colour of the screen when it shrinks down to a certain size, using the **@media** followed by the type of query in parenthesis.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Screen Size</title>
    <style>
      @media (min-width: 500px) {
        body {
          background-color: red;
        }
      }

      @media (max-width: 499px) {
        body {
          background-color: blue;
        }
      }
    </style>
  </head>
  <body>
    <h1>Welcome to the page!</h1>
  </body>
</html>
```

FLEXBOX

Another way is to use the **Flexbox**. This allows us to easily wrap elements around the next line if they don't fit horizontally. We do this by putting all of our elements in a **<div>** (container).

We then add some styling to that **div**, specifying that we want to use a flexbox display for the elements inside it.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Screen Size</title>
    <style>
      #container {
        display: flex;
        flex-wrap: wrap;
      }
      #container > div {
        background-color: green;
        font-size: 20px;
        margin: 20px;
        padding: 20px;
        width: 200px;
      }
    </style>
  </head>
  <body>
    <div id="container">
      <div>Some text 1!</div>
      <div>Some text 2!</div>
      <div>Some text 3!</div>
      <div>Some text 4!</div>
      <div>Some text 5!</div>
      <div>Some text 6!</div>
      <div>Some text 7!</div>
      <div>Some text 8!</div>
      <div>Some text 9!</div>
      <div>Some text 10!</div>
      <div>Some text 11!</div>
      <div>Some text 12!</div>
    </div>
  </body>
</html>
```

GRID

Another popular way of styling a page is using an HTML grid. In this grid, we can specify style attributes such as column widths and gaps between columns and rows, as demonstrated below. Note that when we specify column widths, we can say the third one is auto, meaning it should fill the rest of the page. (https://www.w3schools.com/css/css_grid.asp)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>My Web Page!</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <style>
      .grid {
```

```

        background-color: green;
        display: grid;
        padding: 20px;
        grid-column-gap: 20px;
        grid-row-gap: 10px;
        grid-template-columns: 200px 200px auto;
    }
    .grid-item {
        background-color: white;
        font-size: 20px;
        padding: 20px;
        text-align: center;
    }
</style>
</head>
<body>
    <div class="grid">
        <div class="grid-item">1</div>
        <div class="grid-item">2</div>
        <div class="grid-item">3</div>
        <div class="grid-item">4</div>
        <div class="grid-item">5</div>
        <div class="grid-item">6</div>
        <div class="grid-item">7</div>
        <div class="grid-item">8</div>
        <div class="grid-item">9</div>
        <div class="grid-item">10</div>
        <div class="grid-item">11</div>
        <div class="grid-item">12</div>
    </div>
</body>
</html>

```

BOOTSTRAP

It turns out that there are many libraries that others have already written that can simplify the styling of a webpage. One popular library that we'll use throughout the course is known as bootstrap (<https://getbootstrap.com/>)

We can include Bootstrap in our code by adding a single line to the head of our HTML file:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
T3c6CoLi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwvykc2MPK8M2HN"
crossorigin="anonymous">
```

Next, we can look at some of Bootstrap's features by navigating to the **documentation** portion of their website. On this page (<https://getbootstrap.com/docs/4.5/components/>), you'll find many examples of classes you can add to elements that allow them to be styled with Bootstrap.

Read more at

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>